High-Level Design Document

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

Project Type	Internship Deliverable – Deliverable 1 of 3
Document Type	High-Level Design (HLD)
Technology Stack	Python · LangGraph · ChromaDB · Google Gemini

1. System Overview

1.1 Problem Definition

Modern customer support teams face a compounding challenge: they must scale rapidly while still delivering accurate, context-aware answers in near-real-time. Standard Large Language Models (LLMs) can generate fluent, human-like text but are fundamentally limited in two important ways.

- **Hallucination risk.** LLMs may confidently produce incorrect answers when operating outside their training distribution — a critical failure mode in customer-facing deployments.
- **No access to proprietary knowledge.** Internal product manuals, policy documents, and company-specific FAQs are not part of any public model's training data and cannot be injected through a simple system prompt at scale.
- **Limited escalation paths.** Fully autonomous bots have no graceful fallback for genuinely ambiguous or sensitive queries, which erodes user trust.

This project addresses all three limitations by combining a structured retrieval pipeline with a stateful graph-based workflow and a human escalation mechanism — resulting in a system that is both accurate and auditable.

1.2 Scope of the System

The system is scoped to the following core capabilities for this internship deliverable. Each item maps directly to a mandatory concept from the project brief.

PDF Knowledge Base Ingestion	Processing and indexing a proprietary PDF document into a searchable vector store.
Dense Vector Retrieval	Using embedding models to fetch contextually relevant document chunks for any incoming user query.
Stateful Graph Orchestration	Managing the end-to-end query lifecycle through a LangGraph-powered workflow that can branch, loop, and pause.
Intent-Based Routing	Dynamically directing the query to the LLM path or the escalation path depending on the retrieved context quality.
Human-in-the-Loop (HITL) Escalation	Pausing the autonomous workflow and routing unanswerable queries to a live human agent without breaking the user experience.

2. Architecture Diagram

The system is divided into two distinct execution phases. The offline **Document Ingestion Pipeline** runs once (or on-demand) to build the knowledge base. The online **Query Execution & Workflow Pipeline** handles every real-time user interaction.

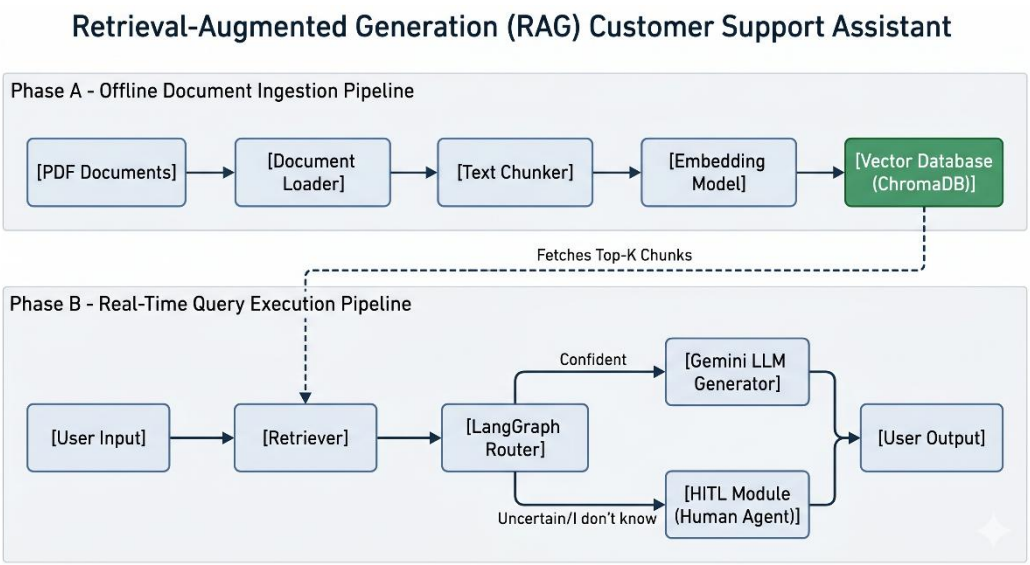

Figure 1 – Logical system architecture (top-to-bottom flow).

3. Component Description

Component	Detail
Document Loader	Reads the raw PDF file from disk using PyPDF and extracts plain string text from each page. The output is a single unstructured text corpus ready for the chunking stage.
Chunking Engine	Implements a **recursive character text splitter** strategy. Large documents are divided into overlapping logical segments — 1,000 characters per chunk with a 200-character overlap. The overlap is critical: it preserves semantic continuity across chunk boundaries and prevents context from being cut mid-sentence.
Embedding Model	Converts each string chunk into a high-dimensional dense vector using **Google's gemini-embedding-001** model. These vectors encode the semantic meaning of the text, enabling similarity-based comparison rather than keyword matching.
Vector Store	A specialised database (**ChromaDB**) optimised for storing and querying high-dimensional vectors. It persists embedding data to disk and exposes rapid approximate nearest-neighbour (ANN) search capabilities.
Retriever	At query time, the incoming user question is embedded using the same model and compared against all stored vectors. The Retriever returns the **Top-K most semantically similar chunks** (K=4 by default) to be used as context for the LLM.
LLM — Generative Engine	Google's **gemini-2.5-flash** model ingests a structured prompt containing the user's query and the retrieved context. It is instructed to answer *only* from the provided data and to return a clearly formatted signal when the context is insufficient.
Graph Workflow Engine (LangGraph)	Manages the overall execution logic as a **cyclic, stateful directed graph**. It holds a shared State object containing the query, retrieved chunks, generated answer, and routing flags. Unlike linear chains, LangGraph supports pause-and-resume behaviour, making HITL feasible.
Routing Layer	Evaluates conditional edges within the graph after the LLM node runs. If the LLM returned a valid answer, the graph routes to the Output node. If the LLM flagged an inability to answer, the graph routes to the HITL node.

HITL Module	A **pause-and-resume interrupt** inside the LangGraph state machine. When triggered, it halts autonomous execution, presents the escalated query to a human agent via the CLI/UI, accepts their typed response, and injects it back into the graph's State to be returned to the user.

4. Data Flow

4.1 Ingestion Phase (Runs Once)

Before the system can answer any queries, the PDF knowledge base must be processed and indexed. This phase executes offline and its output is persisted to disk so it does not need to repeat on every restart.

1	The Document Loader reads the PDF file and extracts raw text from each page.
2	The Chunking Engine splits the full text into overlapping segments (1,000 chars, 200-char overlap).
3	Each chunk is passed to the Embedding Model, which returns a numeric vector representing its meaning.
4	The (chunk text + vector) pairs are written to ChromaDB and saved to disk.

4.2 Query Lifecycle (Real-Time)

Every time a user submits a question, the following sequence is executed within the LangGraph state machine.

User Input: A plain-text question is received via the CLI or web interface and stored in the graph's State object.

Query Embedding: The question is converted into a vector using the same embedding model used during ingestion, ensuring the vector space is consistent.

Similarity Retrieval: The Retriever queries ChromaDB for the Top-4 chunks whose vectors are closest to the query vector. These become the LLM's context window.

Prompt Construction: A strict prompt template combines the user query and the retrieved chunks. The LLM is instructed not to speculate beyond the provided data.

LLM Generation & Routing: The LLM attempts to generate an answer. The Routing Layer inspects the output: a valid answer proceeds to the Output node; an 'I don't know' signal triggers the HITL node.

HITL Escalation (conditional): The graph pauses. A human agent is notified, reviews the original query, and types a response. Their input is injected into State.

User Response: The final answer — whether AI-generated or human-provided — is returned to the user interface.

5. Technology Choices

5.1 ChromaDB — Vector Database

ChromaDB is an open-source, lightweight vector store that runs natively in Python with zero external infrastructure. For a prototyping and internship context, this is the right tool: it eliminates cloud setup overhead, persists data to a local directory, and integrates seamlessly with LangChain. Its cosine similarity search is performant for knowledge bases of moderate size.

5.2 LangGraph — Workflow Orchestration

Standard LangChain pipelines are linear — data flows in one direction and there is no mechanism to pause execution mid-workflow. LangGraph introduces the concept of a stateful, cyclic directed graph where individual nodes can modify a shared State object and conditional edges can branch the flow. This architecture is architecturally mandatory for HITL: the graph must be able to halt, wait for external human input, and then resume — something a simple pipeline cannot do.

5.3 Google Gemini (Free Tier) — LLM & Embeddings

The project uses two Google Gemini models. **gemini-2.5-flash** handles answer generation: it offers strong instruction-following, is fast, and performs well under strict prompt constraints. **gemini-embedding-001** produces dense vector embeddings that are semantically rich and consistent across both ingestion and retrieval. Both models are available on the free tier, making them appropriate for internship-level development and testing.

5.4 Additional Tools

Component	Detail
PyPDF	Reliable, lightweight library for extracting plain text from PDF files.
langchain_core	Provides robust prompt template abstractions and manages structured interactions with the Gemini API.
Python (3.10+)	The runtime environment. All components are Python-native, ensuring a single-language codebase with minimal dependency friction.

6. Scalability Considerations

6.1 Handling Large Document Volumes

The current architecture ingests a single PDF synchronously. As the knowledge base grows — potentially to hundreds of product manuals or policy documents — two changes become necessary. First, the ingestion pipeline should be decoupled into an asynchronous background job so that new document uploads do not block the query-serving path. Second, chunk metadata (e.g., document

source, product category, creation date) should be stored alongside each vector to enable pre-filtering at retrieval time, reducing unnecessary similarity comparisons.

6.2 Increasing Query Load

A local ChromaDB instance is not designed for concurrent multi-user access. Under production load, the recommended migration path is to a cloud-managed vector database — such as Pinecone or hosted Milvus — which provides horizontal scaling, replication, and managed backups. The application logic layer (the LangGraph worker) should be containerised with Docker and deployed behind a load balancer (e.g., Kubernetes) to handle parallel query execution.

6.3 Latency Optimisation

For real-time responsiveness, three strategies are recommended for a production deployment. **Semantic caching** stores previous query-answer pairs; if an incoming query is semantically similar to a cached one, the LLM call is bypassed entirely. **Model tiering** uses a smaller, fine-tuned model for the routing decision (which is a simple binary classification) rather than relying on the full Gemini Flash model for this lightweight task. **Async retrieval** runs the vector search and prompt construction concurrently where possible to reduce end-to-end wall-clock time.